

Lesson2: Basics of Python



Today's Goals



- Understand why we learn Python
- Understand the basic grammar of Python
- Use Google Colaboratory (necessary for the future GCI classes)

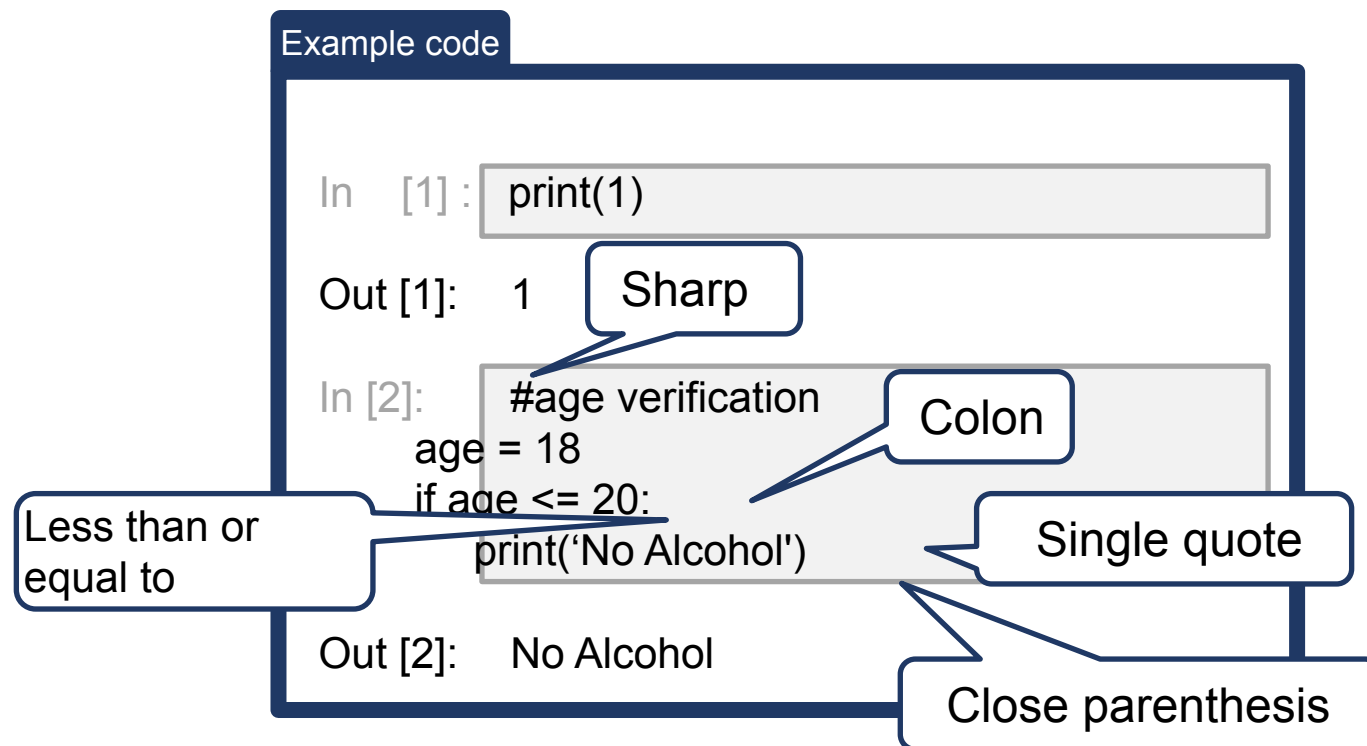
Not all grammar items will be covered in this lecture.
For more details, please take a look at our pre-learning materials (GCI Basic).

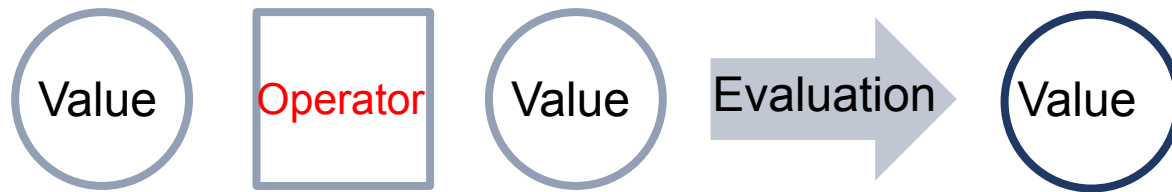
- Ease of use and readability
- Extensive libraries
- Frequent use in academic research
- Open source and extensibility
- High demand in the job market

- Don't try to perfect everything at once
- If you don't understand, ask google or ChatGPT etc.
- Ask on our Slack workspace
 - When asking...
 - Present code and error messages and ask questions about how to correct them
- Visit the official website
 - <https://docs.python.org/3/reference/introduction.html>

Open Google Colab

How to read Python programs





Example code

```
In [1]: print(1 + 1)
```

```
Out [1]: 2
```

```
In [2]: print(1 - 1)
```

```
Out [2]: 0
```

Example Arithmetic Operator: Addition



Arithmetic operations	Operator symbols
Addition	+
Subtraction	-
Multiplication	*
Division	/
Division (quetient)	//
Division (remainder)	%
Powers	**

1×2

$1 \div 2$

2^2

Example code

In [1]:

print(1 * 2)

Out [1]:

2

In [2]:

print(1 / 2)

Out [2]:

0.5

In [3]:

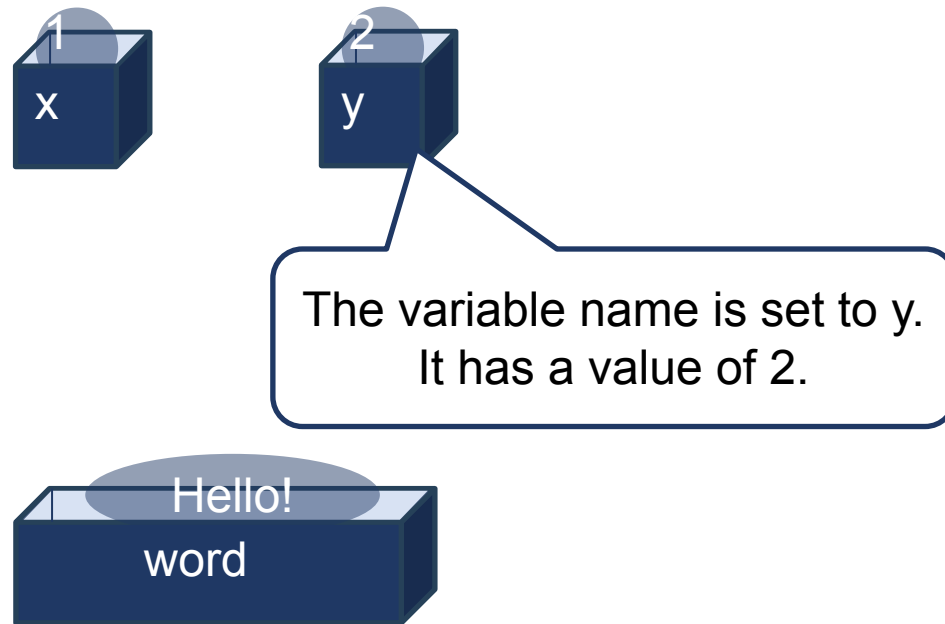
print(2 ** 2)

Out [3]:

4

Variables

By setting **variable name = value**, the data on the right side is assigned to the variable on the left side.



Example code

```
In [1]: x = 1  
        y = 2  
        print(x+y)
```

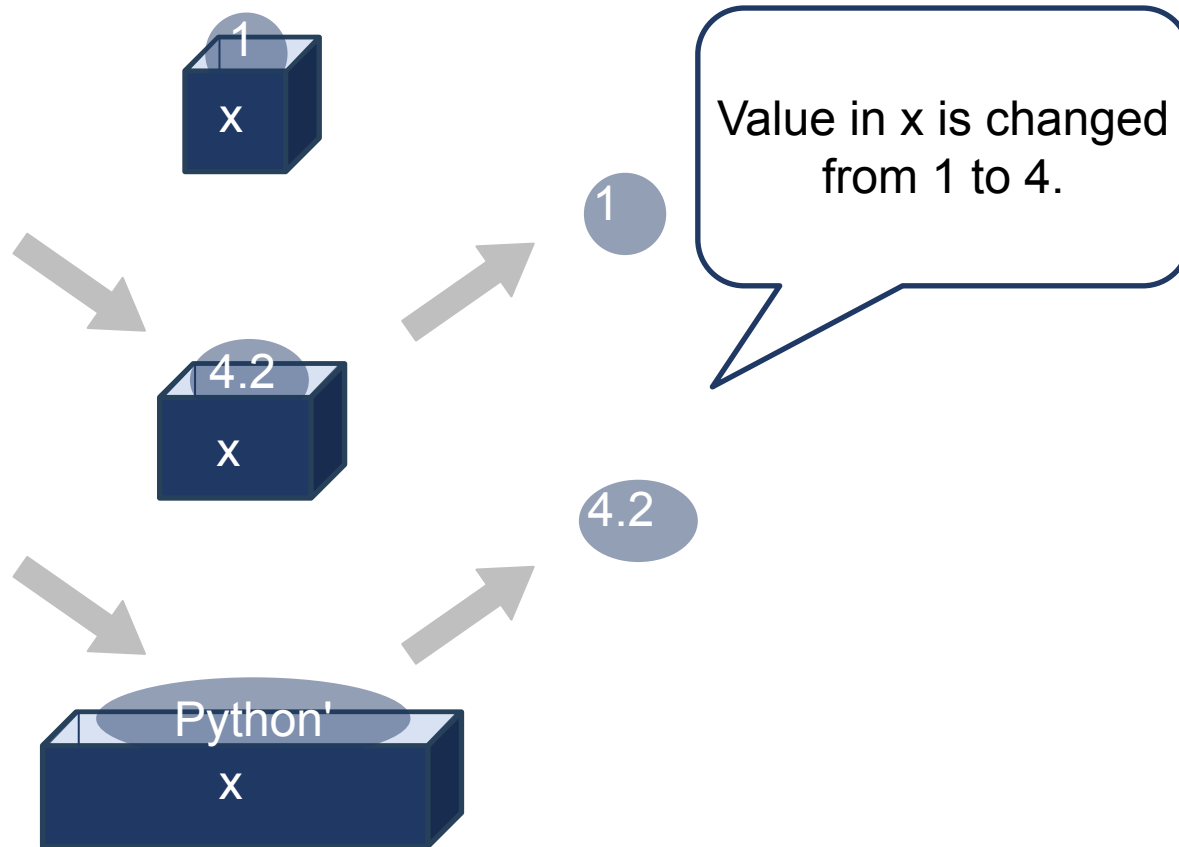
Out [1]: 3

```
In [2]: word = 'Hello'  
        print(word)
```

Out [2]: Hello!

Variables

By **reassigning** to the same variable, you can change the stored value.



Example code

```
In [1]: x = 1  
        print(x)
```

```
Out [1]: 1
```

```
In [2]: x = 4.2  
        print(x)
```

```
Out [2]: 4.2
```

```
In [3]: x = 'Python'  
        print(x)
```

```
Out [3]: Python
```

Data types	Contents	Examples
Int	Integer	1
float	Decimal (Floating Point)	3.14
str	String	'Python'
bool	True or false value	True or false

Example code

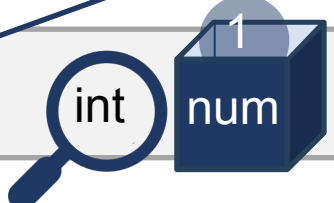
```
In [1]: num = 1
        print(type(num))

Out [1]: int
```

```
In [2]: pi = 3.14
        print(type(pi))

Out [2]: float
```

The type function outputs the datatype of the input

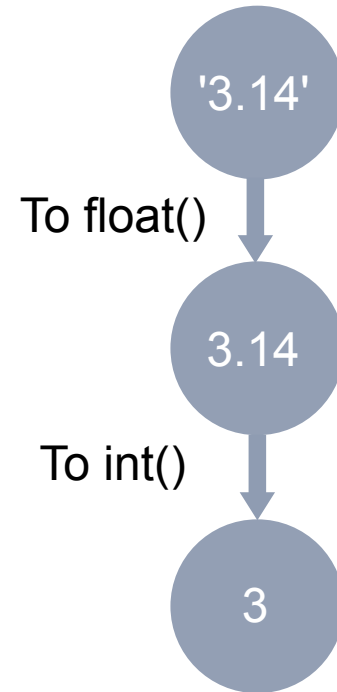


Type Conversion



The data type of the value assigned to a variable can be changed with int function, str function, etc.

Function	Output type
int()	Integer to int type (*1)
float()	To float type of decimals (*2)
To str()	To str type of string
To bool()	To str type of bool



Example code

```
In [1]: x = '3.14'
        print(type(x))
```

```
Out [1]: str
```

```
In [2]: y = float(x)
        print(type(y))
```

```
Out [2]: float
```

```
In [3]: z = int(y)
        print(z)
```

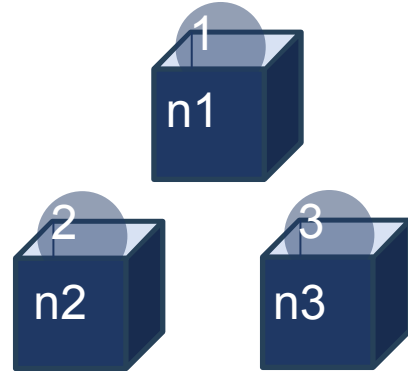
```
Out [3]: 3
```

[Open Google Colab](#)

Collections

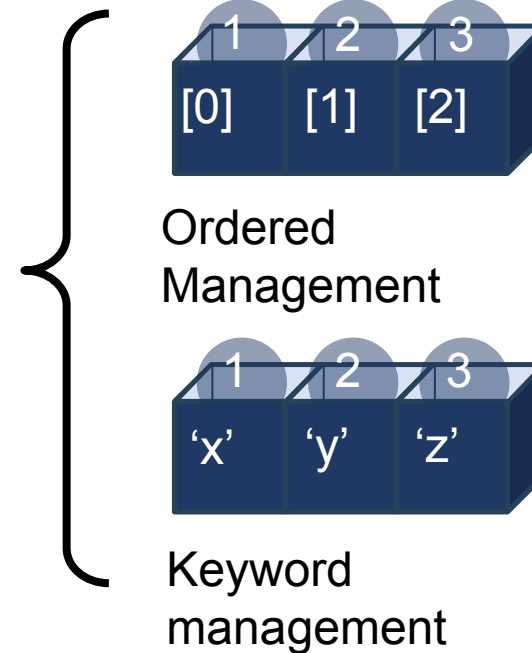
Collections allow you to manage values together.
There are types of **lists**, **tuples**, **dictionaries**, etc.

Assigning Values to Each Variable



Summary

Assigning Values to Collections



Example

- Lists
- Tuples

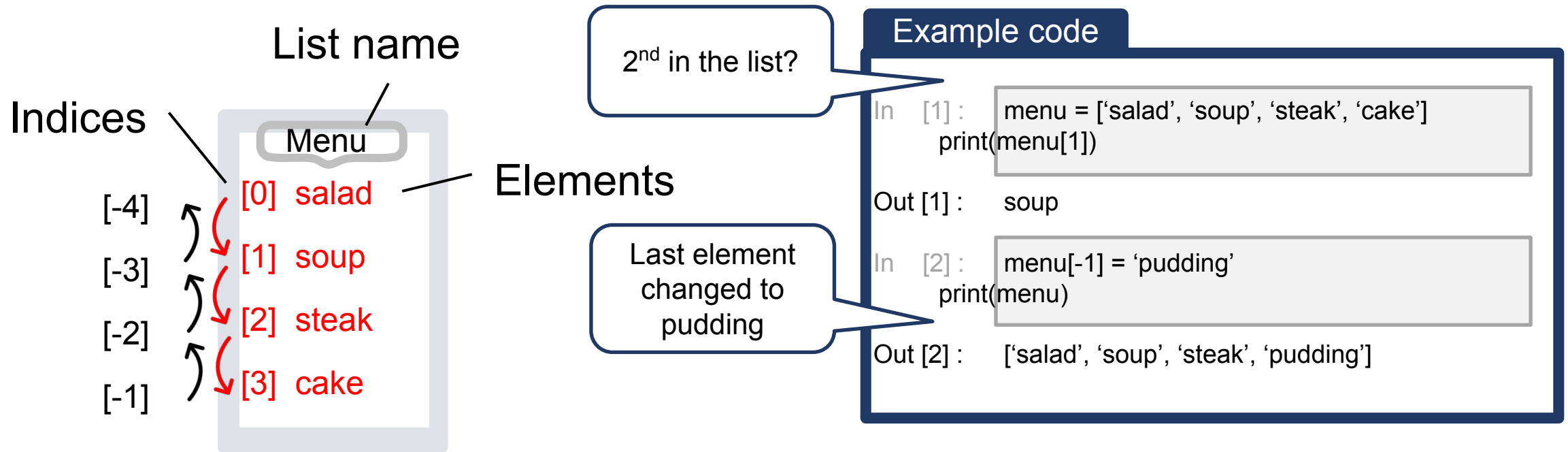


- Dictionaries



List

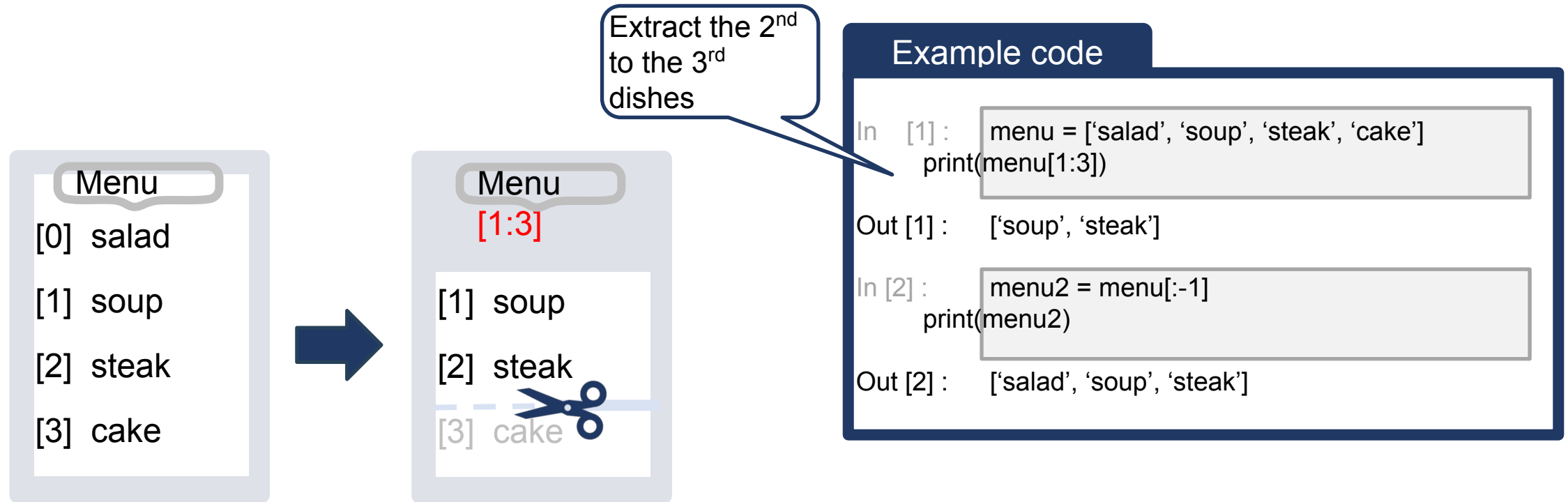
Lists use **brackets []** to manage values (elements) in order (indices).
An elements can be extracted and replaced by specifying the index.



Indexes start at zero, and from the back they count from minus one.
List elements can be of data types such as int or float in addition to str type.

Slicing a list

You can slice a list out of the list by specifying an index range.



Slice notation `[x:y]` represents `x` or more but less than `y`, and if `x` or `y` is omitted, the remaining range is selected.

Appending to or removing from a list

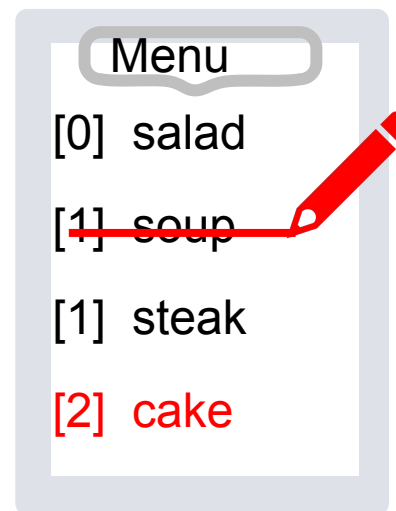
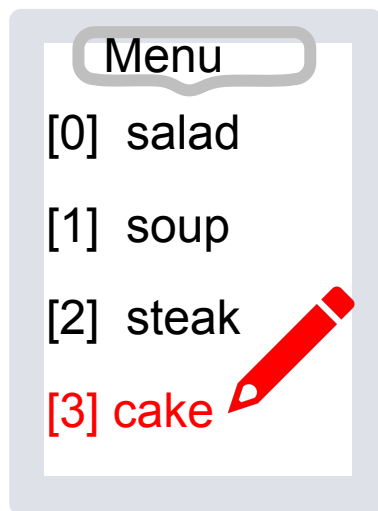
Example code

```
In [1]: menu = ['salad', 'soup', 'steak']  
        menu.append('cake')  
        print(menu[-1])
```

Out [1]: cake

```
In [2]: menu.remove('soup')  
        print(menu[-1])
```

Out [2]: ['cake']



Numerical Calculations on Lists

There are built-in functions for calculating the number of lists.

Functions	Output
<code>len()</code>	Number of elements
<code>sum()</code>	Total value
<code>max()</code>	Maximum value
<code>min()</code>	Minimum value

Example code

```
In [1]: score = [40, 60, 80]
        print(sum(score) / len(score))
```

```
Out[1]: 60
```

```
In [2]: max_score = max(score)
        print(max_score)
```

```
Out[2]: 80
```

Two-dimensional list

Two-dimensional tabular data can also be managed using **double brackets**.

	Column 0	Column 1	Column 2
Column 0	['24/3',	'24/4',	'24/5',
Row 1	[80,	60,	90]]

Elements with the 1st row and the 2nd column
[0][1]

Example code

```
In [1]: table = [['24/3', '24/4', '24/5'], [80, 60, 90]]
        print(table[0][1])

Out [1]: 24/4

In [2]: print(table[1][2])

Out [2]: 0
```

Elements with 2nd row and the 3rd column
[1][2]

Tuple



A tuple is similar to a list, but its order cannot be changed once defined.

Example code

```
In [1]: color = ('red', 'green', 'blue')  
        print(color[0])
```

Out [1]: red

```
In [2]: menu = ('rice', 200)  
        print(menu[1])
```

Out [2]: 200

Can manage elements with
different data types

Difference with a list



Lists [] are **Mutable**: they
can be changed after
creation.



Tuples () are **Immutable**:
they cannot be changed.

Dictionary



Dictionaries use `{}` to hold **data in pairs of keys** and their corresponding **values**.

Example code

```
In [1]: menu = {'apple':100, 'banana':80, 'orange':120}  
        print(menu['apple'])
```

Out [1]: 100

```
In [2]: menu['grape'] = 150  
        print(menu)
```

Add an element whose key is 'grape' and whose value is 150

Out [2]: {'apple':100, 'banana':80, 'orange':120, 'grape':150}

```
In [3]: menu['banana'] = 60  
        print(menu)
```

Key updates the value of 'banana'

Out [3]: {'apple':100, 'banana':60, 'orange':120, 'grape':150}

```
In [4]: menu = {'apple': 100, 'banana': 80, 'orange': 120}  
        del menu['apple']  
        print(menu)
```

Deleting an element with the key

Out [4]: {'banana': 80, 'orange': 120}

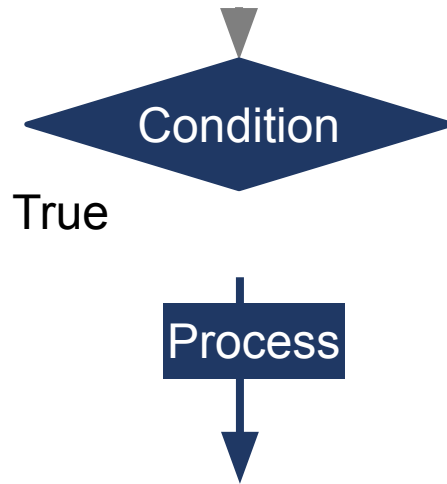
[Open Google Colab](#)

if statement

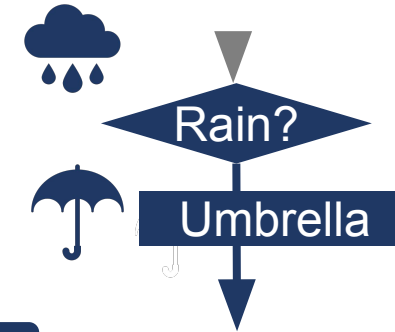
An if statement refers to a way of writing code that changes the flow of processing depending on the **boolean** value of the condition (True or False).

(Example) If it rains, hold an umbrella

(if ... condition ... do)



False



Example code

```
In [1]: is_rain = True
        if is_rain:
            print('Hold an umbrella')
```

```
Out [1]: Hold an umbrella
```

Example code

In [1]:

```
x=1
if x < 0:
    x = -x
    x = -x
print(x)
```

Out [1]: 1

Both lines are
conditional

In [2]:

```
x=1
if x < 0:
    x = -x
x = -x
print(x)
```

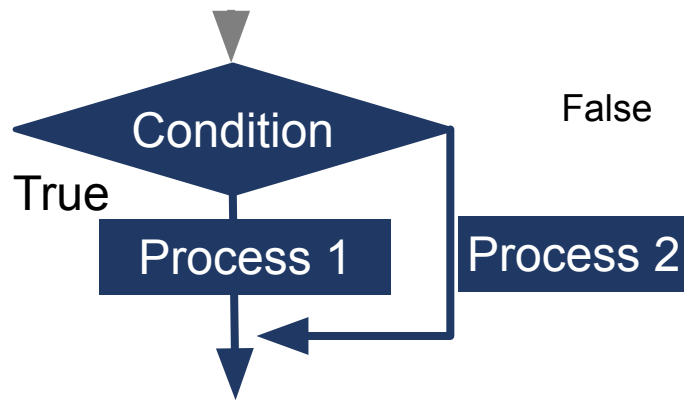
Out [2]: -1

Only the first line is
conditional

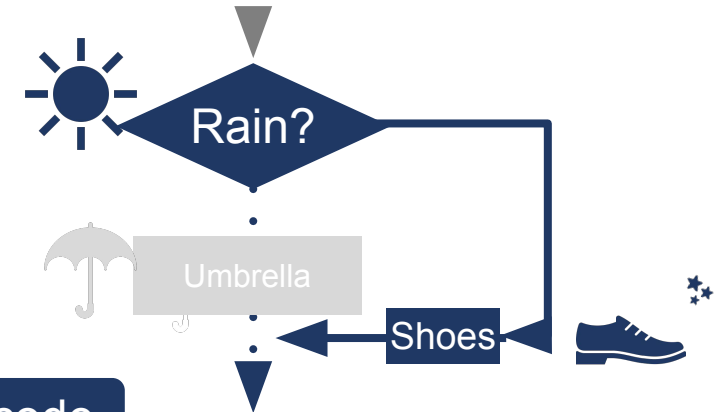
else statement

Use else statement with the if statement to add a process when the condition is false.

(If not...)



(Example) If it rains, hold an umbrella.
Otherwise, wear a new shoe



Example code

```
In [1]: is_rain = False
if is_rain:
    print('Hold an umbrella')
else:
    print('Wear a new shoe')
```

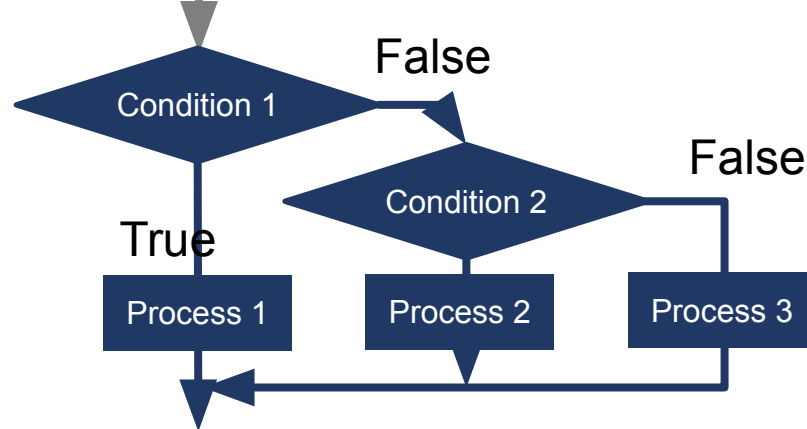
Out [1]: Wear a new shoe

elif statement



If you want to have three or more conditional branches, add elif statement.

If-elif-else statement



Example code

In [1]: BMI = 20

False

If BMI >= 25:

print('overweight')

elif BMI >= 18 and BMI < 25:

print('normal weight')

else:

print('underweight')

True

Out [1]: normal weight

Comparison Operator

Comparison operators compare two values. They evaluate an expression and return a **Boolean value**: either True or False.

Operator symbols	Meaning
==	Left and right values are the same
!=	Left and right values are not the same
>	Left values are larger than right values
<	Left values are smaller than right values
>=	Left is greater than or equal to the right value
<=	Left is less than or equal to the right value

Example code

```
In [1]: print(1 == 1)
```

```
Out [1]: True
```

```
In [2]: print(1 < 0)
```

```
Out [2]: False
```

Example: Comparison operator: <



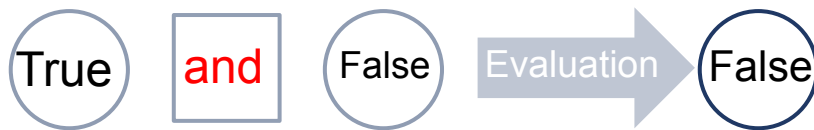
Logical Operator

Logical operators evaluate boolean values of multiple operators.
They are often used in combination with comparison operators in an if statement

Operator symbol
and
or
not

Operators	Priority
Arithmetic	High
Comparison	Medium
Logic	Low

“And” operator



“Not” operator



Example code

```
In [1]: print(True and False)
```

```
Out [1]: False
```

```
In [2]: print(True or False)
```

```
Out [2]: True
```

```
In [3]: BMI = 20  
        If BMI >= 18 and BMI < 25:  
            print('normal weight')
```

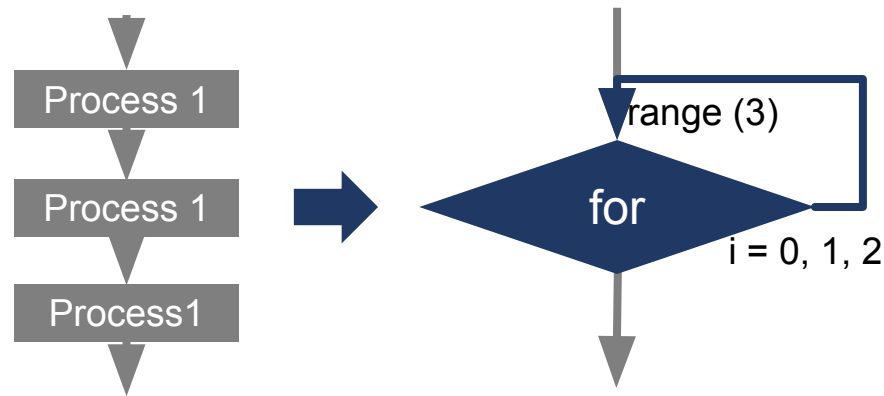
```
Out [3]: normal weight
```

True and True ➤ True

for Statement



Used when you want to repeat the same process without writing it over and over again. You can specify ranges and repeat them.



Example code

```
In [1]: for i in range(5):  
        print('Pythongrammar', i)
```

```
Out [1]: Python grammar 0  
         Python grammar 1  
         Python grammar 2  
         Python grammar 3  
         Python Grammar 4
```

for Statement & List



If you combine a list with a for statement, you can extract each element and repeat the same process.

Example code

```
In [1]: menu = ['salad', 'soup', 'steak', 'cake']
```

```
for food in menu:  
    print(food)
```

```
Out [1]:  salad  
         soup  
         steak  
         cake
```

```
In [2]: menu = ['salad', 'soup', 'steak', 'cake']
```

```
for i, food in enumerate(menu):  
    print(i, food)
```

```
Out [2]:  0 salad  
         1 soup  
         2 steak  
         3 cake
```

Menu

[0] salad

[1] soup

[2] steak

[3] cake



Use the "enumerate" function to extract both the elements and the indices

[Open Google Colab](#)

DRY (Don't Repeat Yourself)

DRY is a software design principle that keeps each piece of knowledge or logic in a single authoritative location to avoid duplication. This reduces the chance of inconsistencies and makes maintenance simpler.

Example code

Duplicate Code

```
r1 = 2
area1 = 3.14159 * r1 * r1

r2 = 3
area2 = 3.14159 * r2 * r2
```

DRY

```
PI = 3.14159

def area_circle(r):
    return PI * r * r

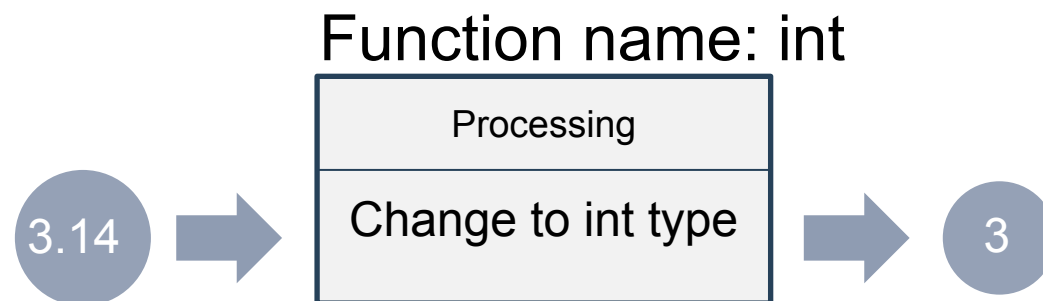
area1 = area_circle(2)
area2 = area_circle(3)
```

Function

A function in Python is a reusable block of code that performs a specific task when called. It can take inputs (**arguments**) and can return an output (**result**).

Functions	Details of Processing
print()	Show Values
type()	Output Data Types
int()	Change to integer (int) type
str()	Change to string (str) type
sum()	Output the total value
len()	Output the Number of Elements

Example: the “int” function



Function



By defining your own function, you can use the same process repeatedly without having to write the code over and over again.

Example code

```
In [1]: def greet():  
        print("Hello")
```

```
greet()
```

```
Out [1]: Hello.
```

```
In [2]: greet()  
        greet()
```

```
Out [2]: Hello.  
        Hello.
```

```
In [2]: def greet(name):  
        print('Hello,' name)
```

```
greet('Satoshi')
```

```
Out [2]: Hello, Satoshi.
```

```
In [3]: greet('Pikachu')
```

```
Out [3]: Hello, Pikachu.
```

Passing an
argument

Defining the **return**
value

```
In [1]: def greet(name):  
        string = 'Hello,' name  
        return string
```

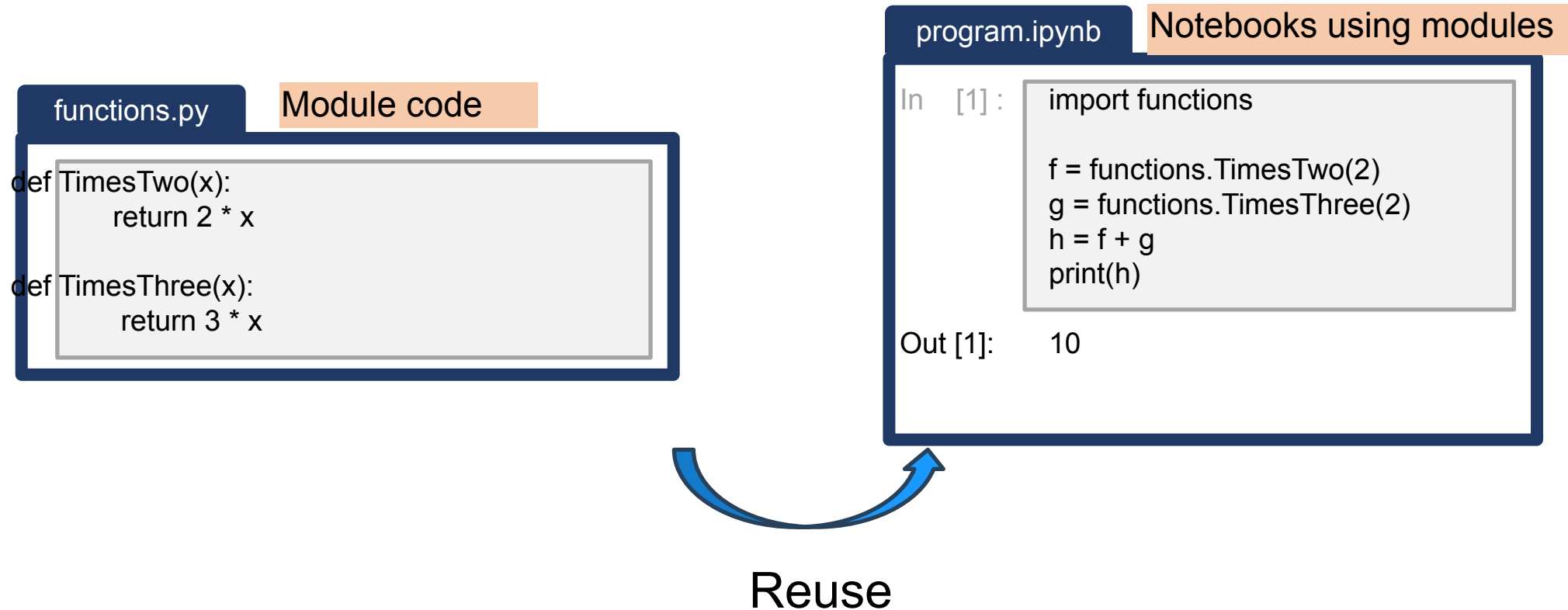
```
string = greet('Satoshi')  
print(string)
```

```
Out [1]: Hello, Satoshi
```

Module



A **module** is a file containing Python definitions and statements.



A **library** is a collection of modules packaged together to provide specific functionality.

Example code

In [1]: `import numpy`

`x = numpy.array([1, 2])`

Importing a library named "numpy"

In [2]: `import numpy as np`

`x = np.array([1, 2])`

Abbreviation

In [3]: `import math`

`print(math.pi)`

Out [3]: 3.141592653589793

In [4]: `from math import pi`

`print(pi)`

Out [4]: 3.141592653589793

You can use **"from"** to import specific elements from a module or library, allowing you to use them directly without prefixing them with the module name.

Class



A class is a blueprint for creating objects that bundle data and functions together.

Example code

```
In [1]:
class Pokemon:
    def __init__(self, name, hp):
        self.name = name
        self.hp = hp

    def take_damage(self, amount):
        self.hp = self.hp - amount
        print(f"{self.name} took {amount} damage! HP: {self.hp}")

pikachu = Pokemon("Pikachu", 100)
charmander = Pokemon("Charmander", 90)

pikachu.take_damage(10)

Out [1]:
Pikachu took 10 damage! HP: 90
```

The constructor method

`__init__` . Called when object is created

What is `self`?

The current object.

Access Attributes

`variable_name.attribute_name`

Create an Object (**Instance**)

`variable_name = ClassName()`

How to Use **Methods**

`variable_name.method_name()`

[Open Google Colab](#)

Grammars that were not covered in today's lecture but will appear in future lectures:

- Anonymous function (lambda expression)
- Map function
- List comprehension
- Conditional expression (ternary operator)
- Iterator

If you're a beginner, no worries — today's lecture should be just what you need!